

MODELS AND ROBOTICS SECTION

MARS

Open Projects 2026



ARTIFICIAL INTELLIGENCE

MACHINE LEARNING

WEB DEVELOPMENT

Problem Statements & Deliverables

PART I

Artificial Intelligence & Machine Learning

Problem Statements

01

PROBLEM STATEMENT 1

Support Integrity Auditor (SIA)

Artificial Intelligence / Machine Learning

NLP

CRM Systems

§ 1 BACKGROUND

In enterprise-scale CRM ecosystems, manual ticket triage is riddled with agent fatigue bias, customer favoritism, and keyword anchoring. When critical issues are mislabeled as "Low" or trivial complaints are inflated to "Critical," Service Level Agreements (SLAs) are jeopardized, and customer churn increases. Existing rule-based or keyword-matching systems fail to detect the nuanced discrepancies between a ticket's true severity and its assigned priority.

This project addresses a fundamentally harder variant of the triage problem: there are no pre-annotated mismatch labels. The system must bootstrap its own supervision signal from raw ticket data alone.

§ 2 PROBLEM STATEMENT

Build the Support Integrity Auditor (SIA) — a semantics-driven, evidence-grounded automated auditor that detects *Priority Mismatch*: cases where the objective characteristics of a support ticket (text, customer domain, channel, resolution time) conflict with its human-assigned priority level.

THE SYSTEM MUST:

- Infer the "true" severity of a ticket independent of its assigned label.
 - Generate its own binary mismatch supervision signal (self-supervised).
 - Train a fine-tuned classifier on that pseudo-labeled data.
 - Produce a structured, hallucination-free Evidence Dossier for every flagged ticket.
 - Generalize to previously unseen tickets, including adversarially crafted examples.
-

§ 3 DATASET

Recommended Dataset: *Customer Support Tickets — CRM Dataset*

Dataset Link: kaggle.com/datasets/ajverse/customer-support-tickets-crm-dataset/data

KEY COLUMNS USED

COLUMN	ROLE
Ticket Subject	Short summary of the issue
Ticket Description	Full natural language problem statement
Customer Email / Product Purchased	Proxy for customer tier / domain
Ticket Priority	Human-assigned label (Low / Medium / High / Critical)
Ticket Channel	Intake channel (email, chat, phone, social media)
Resolution Time	Time to resolve — indirect severity signal
Ticket Type	Category of issue

§ 4 MANDATORY PIPELINE STAGES

STAGE 1: PSEUDO-LABEL GENERATION (SELF-SUPERVISED)

Construct a reproducible pipeline that assigns each ticket an inferred severity, independent of the Ticket Priority column. A binary mismatch label is then derived by comparing the inferred severity to the

assigned priority. The pipeline must fuse at least two of the following independent signals:

- **LLM-based zero-shot severity scoring:** Using an open-source model, e.g., Mistral-7B-Instruct or Phi-3-mini.
- **Embedding-based clustering:** e.g., sentence-transformers, for semantic urgency grouping.
- **Resolution-time regression:** Used as a severity proxy.
- **Rule-based NLP features:** Keyword density, negation detection, escalation phrases.

Requirement: Justify your fusion strategy in the README with an ablation showing each signal's individual contribution.

STAGE 2: CLASSIFIER TRAINING (FINE-TUNED MODEL)

Train a supervised binary classifier on the pseudo-labeled data.

- **Model:** Must use a fine-tuned or adapter-trained model (e.g., LoRA on a small LLM, or fine-tuned DeBERTa-v3-small) — not a frozen zero-shot pipeline.
- **Inputs:** Input features must include text fields and at least one structured metadata feature (channel, domain tier, or resolution time).
- **Imbalance:** Class imbalance must be explicitly addressed (e.g., weighted loss, SMOTE, or oversampling).

STAGE 3: EVIDENCE DOSSIER GENERATION

For every ticket classified as a mismatch, generate a structured dossier using the exact schema below:

```
SCHEMA

{
  "ticket_id": "...",
  "assigned_priority": "...",
  "inferred_severity": "...",
  "mismatch_type": "Hidden Crisis | False Alarm",
  "severity_delta": "",
  "feature_evidence": [
    { "signal": "keyword", "value": "...", "weight": "..." },
    { "signal": "resolution_time", "value": "...", "interpretation": "..." }
  ],
  "constraint_analysis": "<2-3 sentence grounded explanation>",
  "confidence": ""
}
```

Hard Rule: Every feature_evidence item must be traceable to a specific field in the input ticket. Any fabricated or unverifiable claim (hallucination) results in immediate disqualification of that test case.

§ 5 EVALUATION METRICS

Models will be evaluated using the following metrics:

- Binary Classification Accuracy (%)
- Macro F1 Score
- Per-Class Recall (both Consistent and Mismatched classes)
- Pseudo-Label Signal Agreement (pairwise agreement between the two chosen signals)

- Dossier Quality (feature grounding, zero hallucination, severity delta calibration, mismatch type accuracy)

Secondary / Bonus: Adversarial robustness test on 10 held-out tickets designed to fool keyword-based systems. Systems scoring $\geq 7/10$ receive a 10% score bonus.

§ 6 VERIFICATION CRITERIA

A submission is considered valid and successfully completed only if all three of the following conditions are satisfied on the held-out evaluation split:

METRIC	MINIMUM THRESHOLD
Binary Classification Accuracy	$\geq 83\%$
Macro F1 Score	≥ 0.82
Per-Class Recall	≥ 0.78 (on both classes)

Submissions failing to meet any single threshold will not be considered verified. Dossiers containing hallucinated evidence will result in disqualification of the affected test cases regardless of classification scores.

§ 7 EXPECTED DELIVERABLES

1. GITHUB REPOSITORY

- ♦ `notebook.ipynb` — Full reproducible pipeline (pseudo-labeling → training → inference).
- ♦ `train_pipeline.py` — Standalone training script.
- ♦ `predict.py` — Inference script accepting a CSV and outputting predictions + dossiers.
- ♦ `README.md` — Methodology, architecture diagram, ablation table, and metric results.
- ♦ `requirements.txt` — Pinned dependencies.

2. STREAMLIT WEB APP (HOSTED URL)

- ♦ Accepts single-ticket form input or batch CSV upload.
- ♦ Returns binary judgment + full Evidence Dossier per ticket.
- ♦ Displays a Priority Mismatch Dashboard with distribution of flagged tickets, mismatch types, and top contributing signals.
- ♦ Includes a severity delta heatmap across ticket categories and channels.